

Reinforcement Learning

- ① Robots → walk
- ② self driving car
- ③ Go, chess games



trial & error
↓
win

maximum
rewards

Agent → Environment
↑
Rewards



Reinforcement Learning

Autonomous

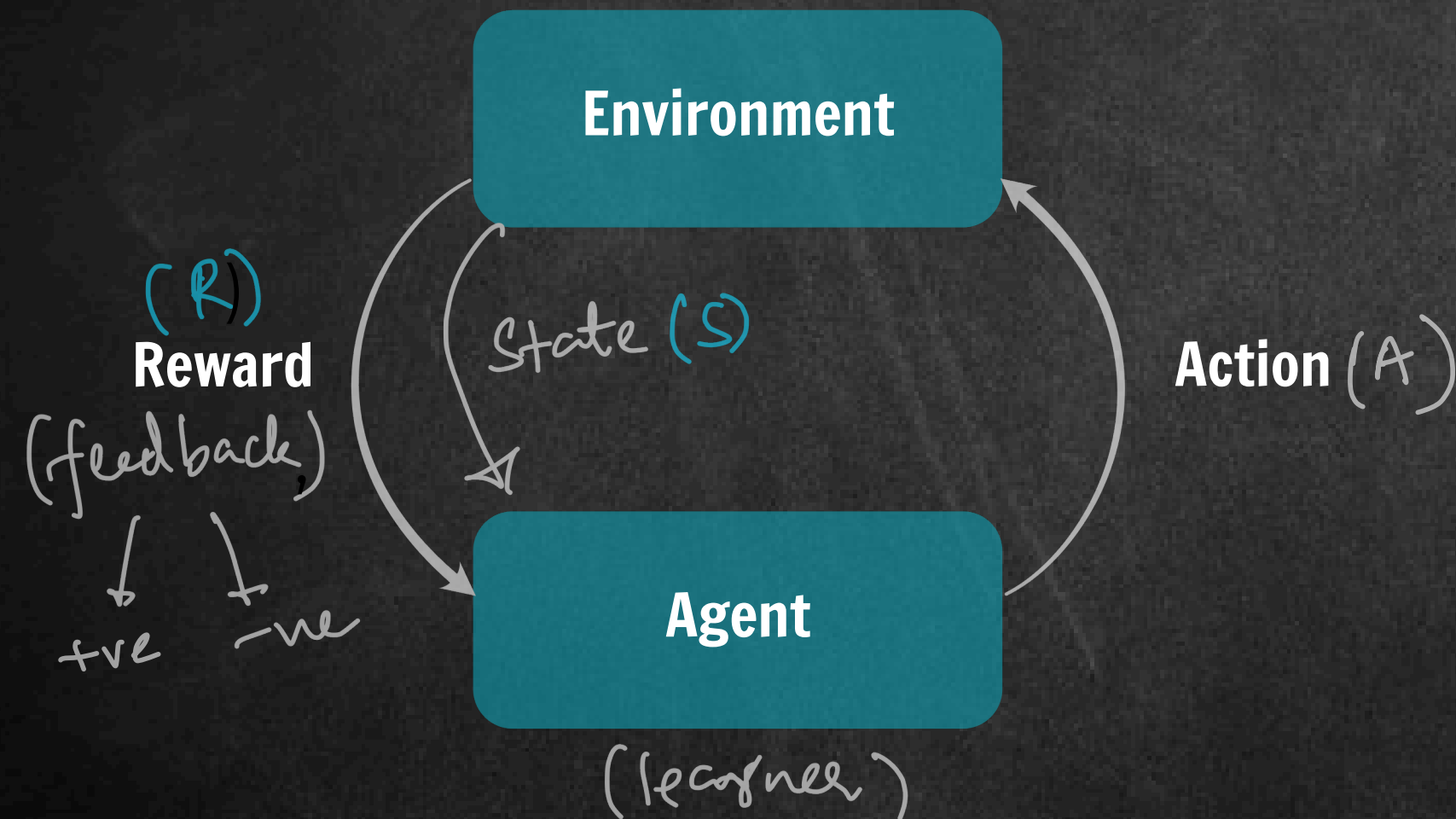
① 2013 DeepMind → Deep Q-Network

RL + DL
↓
integrate

② 2016 Go
↓
AlphaGo → Lee Sedol

Reinforcement Learning

Components



↓
ultimate goal
↓
maximize Rewards

$S \rightarrow$ snapshot of env

$$\textcircled{1} S_t, \textcircled{2} A_t \longrightarrow R_{t+1} \textcircled{3}$$

$\textcircled{4}$ Policy (π) $\rightarrow$ strategy choosing an Action

$\textcircled{5}$ (G) Return (Total future Rewards)
Gain

Reinforcement Learning

Components



Reinforcement Learning

Components

- ① Action (A_t)
- ② Rewards (R_t)
- ③ State (S_t)
- ④ Policy (π) $S \rightarrow A$ $\pi\left(\frac{a}{s}\right)$
- ⑤ Returns (G_t)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$\gamma \rightarrow$ discount factor

$$0 \leq \gamma \leq 1$$

$\gamma=0$ immediate returns

$\gamma=1$ long term

Reinforcement Learning

Components

⑥ Value function (VF)

↓
Returns estimate based on a particular state s .

(a) State VF $\rightarrow V_{\pi}(s)$

estimate return based on state s
followed by policy π

(b) Action VF $\rightarrow Q_{\pi}(a, s)$

max. possible
return of all
policies

- optimal state VF $\rightarrow V_{*}(s)$

- optimal action VF $\rightarrow Q_{*}(a, s)$

Reinforcement Learning

Markov Decision Process (MDP) — math model

↓
sequence of State, Action, Reward

$[S_0, A_0, R_1, \underline{S_1}, A_1, R_2, \dots]$
↓
price
move
↓
✓

$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$

↙
continuous
stock trading
algos.

↓
terminate
check / Go
⇓
Episode

Reinforcement Learning

Markov Decision Process (MDP)

MARKOV property rule

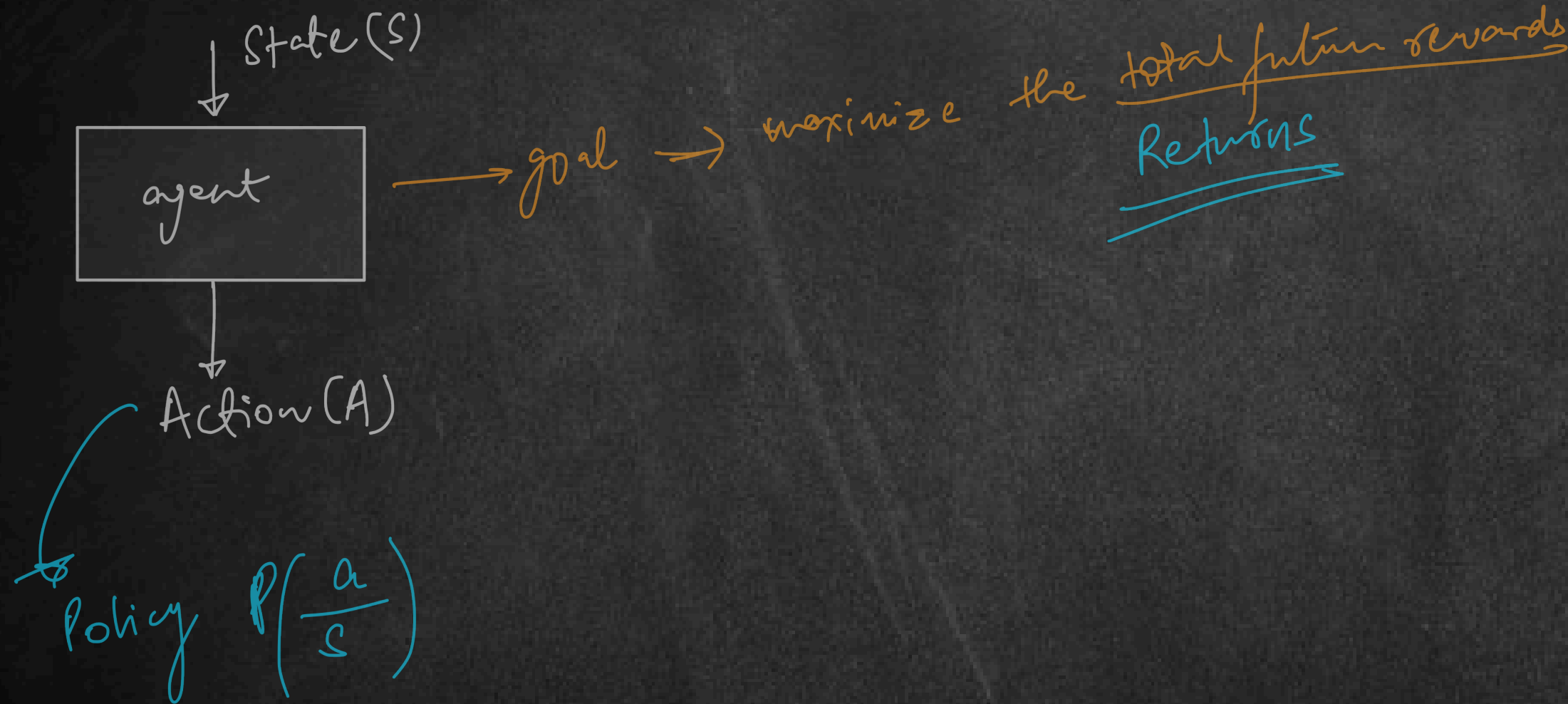
idea: next state $\leftarrow$ cur state, cur action
not the past history

$$S_{t+1} \leftarrow S_t, A_t$$

- limitations $\rightarrow$
- ① not all situation/env fall under MDP
 - ② reward $\rightarrow$ single val

Reinforcement Learning

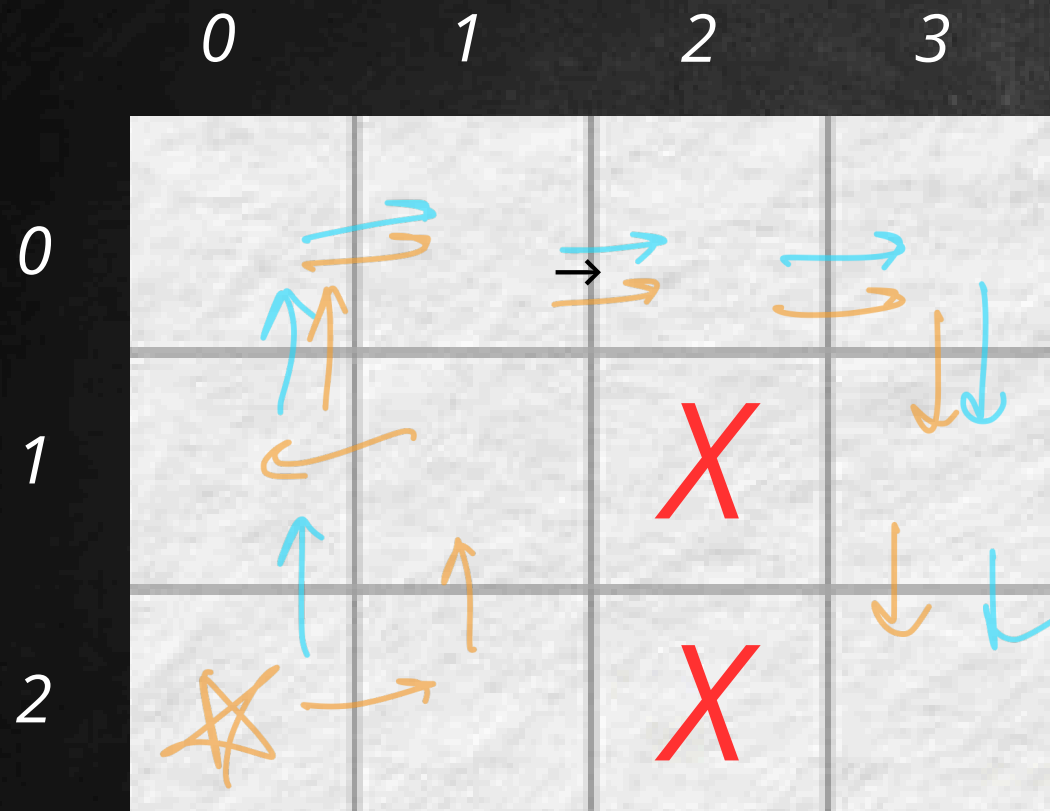
Markov Decision Process (MDP)



- ① Q-learning
- ② SARSA
- ③ Policy Gradients

Reinforcement Learning

Grid Example



allowed moves $\Rightarrow$ up, down, left, right
terminate $\Rightarrow$ E reach OR 15 moves

MDP

$S = \text{cell}(i, j) \rightarrow 2 \text{ nums}$

$A = 0-3 \rightarrow 4 \text{ nums}$

$R = \text{target}(E) \rightarrow 0$

non-E cells $\rightarrow -1$

Reinforcement Learning

Grid Example

	0	1	2	3
0				
1				
2				

-13
-15
9

Policy Optimization

World Model → understand how the env behaves

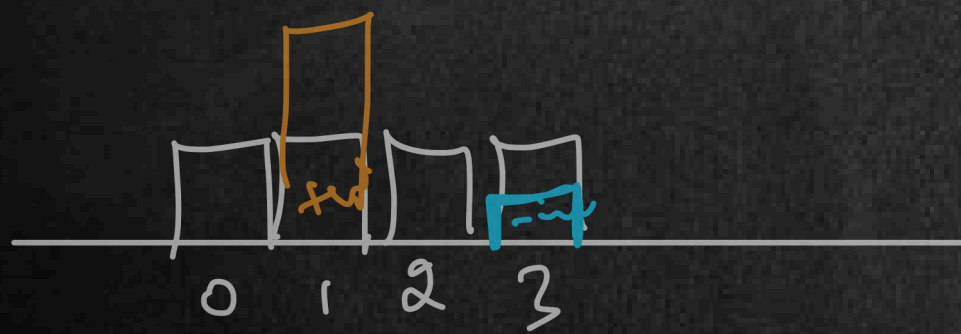
- ① Model based RL
- ② Model free RL

$$P \left(\frac{s_{t+1}, r_{t+1}}{s_t, a_t} \right)$$

Reinforcement Learning

Grid Example

	0	1	2	3
0				
1			X	
2	*		X	E



Policy (π)

Learning

Policy Optimization

Policy Gradient Method

↓
State VF
Action VF

Policy $\rightarrow$ random $(0, 1, 2, 3)$

series of actions $\rightarrow$ better results
 LI " " $\rightarrow$ worse results

actions $\rightarrow$ the
actions $\rightarrow$ -ve

Reinforcement Learning

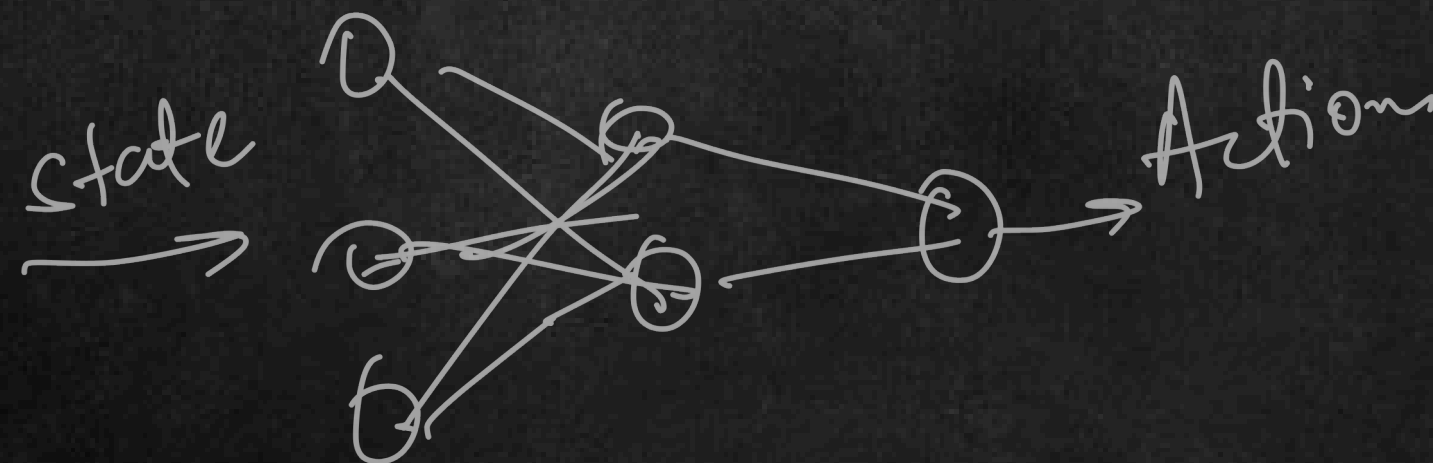
	0	1	2	3
0				
1				
2				

Policy $S \rightarrow A$

State	Action
(2,0)	A_0
0,0	A_1
1,1	A_2
0,2	A_3
	$\vdots$

up $\rightarrow 0$
down $\rightarrow 1$
left $\rightarrow 2$
right $\rightarrow 3$

neural networks



Reinforcement Learning

Grid Example

	0	1	2	3
0				
1				
2				

model free RL : Action-Value $f \rightarrow$ State-Value f

Q-function

$Q_{\pi}(s, a)$

estimate the expected returns of S & A for π

$Q_{*}(s, a)$

pick the highest estimated return



Greedy

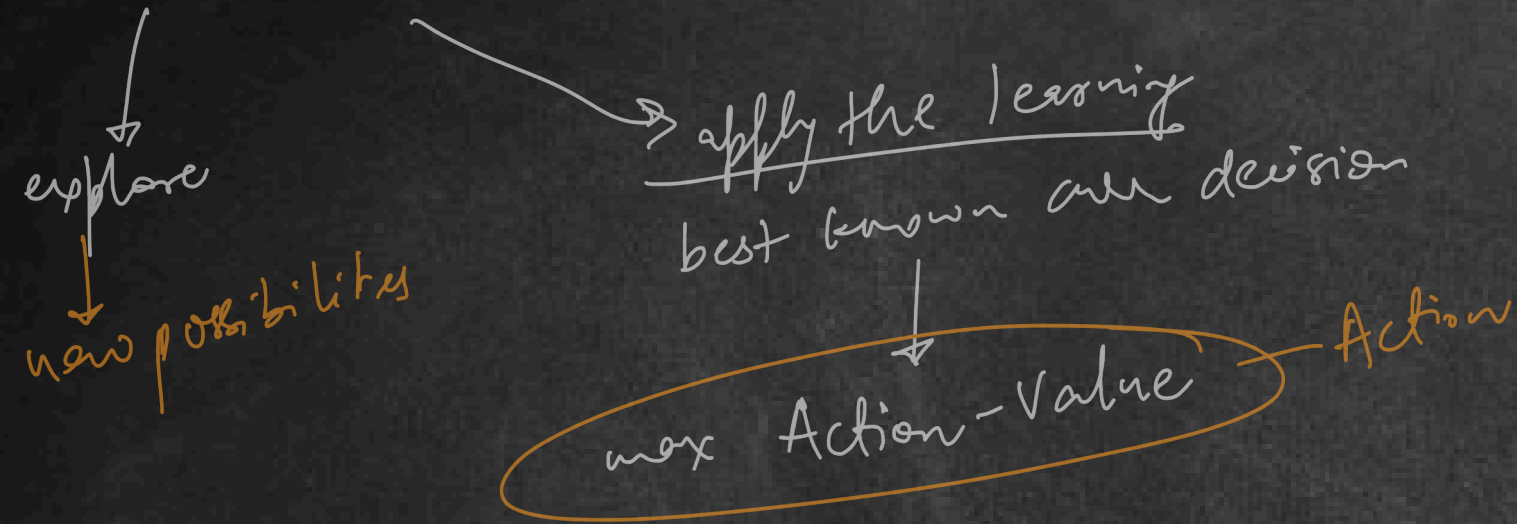
Policy

Q-learning

Deep Q-Networks

Reinforcement Learning

Exploration-Exploitation Tradeoff



ϵ -Greedy Strategy $\rightarrow$ Q-learning

ϵ (probability to explore)

if ($r < \epsilon$)
explore

else
exploit $\Rightarrow$ optimal Q-value

① $\epsilon = 0.1$ (10% chances to explore)

$$0 \leq \epsilon \leq 1$$

②

Action	Q-value
up	5
down	3
left	1
right	10

③

random value (0 to 1) $\rightarrow$ 0

Reinforcement Learning

Exploration-Exploitation Tradeoff

$\epsilon \rightarrow$ gradually decay

probab of explore

start $\rightarrow$ explor $\uparrow$ exploit $\downarrow$
end $\rightarrow$ " $\downarrow$ learning apply $\uparrow$